

DL Midsem Notes — Section 5

Gradient Descent and Loss Surfaces

1 Theory

1.1 Loss / Error Surfaces

Plot the loss $L(w)$ as a function of the parameters w (a surface, or hypersurface for many parameters). Training is the search for the lowest point on this surface. The surface's shape — how many valleys, saddle points, how steep — is determined entirely by the model and data, not by the optimizer; the optimizer just decides how to walk around on it.

1.2 Steepest Descent Direction

At a point w , a first-order (linear) approximation of the loss along a small step $\eta \mathbf{d}$ (unit vector $\mathbf{d}$) is:

$$L(w + \eta \mathbf{d}) \approx L(w) + \eta \nabla L(w) \cdot \mathbf{d}$$

To decrease L as fast as possible, we want $\nabla L(w) \cdot \mathbf{d}$ as negative as possible. By Cauchy-Schwarz, $\nabla L \cdot \mathbf{d} \geq -\|\nabla L\|$, with equality exactly when $\mathbf{d} = -\nabla L / \|\nabla L\|$. So the direction of steepest *decrease* is $-\nabla L$ — this is gradient descent's entire justification (full derivation in Q1).

1.3 The Update Rule

$$w \leftarrow w - \eta \nabla L(w)$$

η (learning rate) controls step size. Too large: overshoot, diverge, or oscillate across a valley. Too small: converges correctly but painfully slowly, and can get trapped for a long time near saddle points/plateaus.

1.4 Local Minima and Saddle Points

A point where $\nabla L = 0$ is a **critical point**: a local minimum, local maximum, or **saddle point** (minimum along some directions, maximum along others). In high dimensions, saddle points vastly outnumber genuine local minima — most of the difficulty of optimizing deep nets in practice is navigating flat regions and saddle points, not escaping bad local minima.

2 Practice Questions

Q1 (PYQ-style, CS6510-2 Q7). Prove that moving in the direction opposite to the gradient minimizes the function (to first order). Then explain the backpropagation weight update rule as an application of gradient descent.

Q2 (numerical). $f(x, y) = x^2 + 3y^2$. Starting at $(2, 1)$ with $\eta = 0.1$, compute one step of gradient descent. What is the new point?

Q3 (concept check). What goes wrong with training if η is too large? What goes wrong if it's too small? Answer in terms of the loss surface picture above.

3 Answers

A1. Proof. For a small step $\eta \mathbf{d}$ ($\mathbf{d}$ a unit vector, $\eta > 0$ small), first-order Taylor expansion gives

$$L(w + \eta \mathbf{d}) \approx L(w) + \eta (\nabla L(w) \cdot \mathbf{d})$$

We want to choose $\mathbf{d}$ to make $L(w + \eta \mathbf{d})$ as small as possible, i.e. minimize $\nabla L \cdot \mathbf{d}$. By Cauchy-Schwarz, $|\nabla L \cdot \mathbf{d}| \leq \|\nabla L\| \|\mathbf{d}\| = \|\nabla L\|$, with the most negative value $-\|\nabla L\|$ attained exactly when $\mathbf{d} = -\nabla L / \|\nabla L\|$ (pointing exactly opposite the gradient). So among all directions, $-\nabla L$ decreases L the fastest, giving

$$L(w - \eta \nabla L) \approx L(w) - \eta \|\nabla L\|^2 < L(w) \quad (\text{for } \nabla L \neq 0)$$

which justifies $w \leftarrow w - \eta \nabla L(w)$.

Backprop connection: backpropagation's entire job is to compute ∇L (one partial derivative per weight) efficiently via the chain rule; the actual parameter update afterward, $w \leftarrow w - \eta \partial L / \partial w$, is exactly this gradient descent step applied per weight.

A2. $\nabla f = (2x, 6y)$. At $(2, 1)$: $\nabla f = (4, 6)$.

$$(x, y) \leftarrow (2, 1) - 0.1(4, 6) = (2 - 0.4, 1 - 0.6) = \boxed{(1.6, 0.4)}$$

A3. Too large: the linear (first-order) approximation the update relies on breaks down over a big step; you can overshoot past the minimum, land on the opposite wall of the valley, and bounce back and forth (or diverge outright, loss increasing each step) instead of settling. **Too small:** each step barely moves along $-\nabla L$, so convergence to even a nearby minimum takes many more iterations than necessary, and progress can stall for a very long time crossing flat regions or near saddle points where $\|\nabla L\|$ is already small.